

packages/graph-core/src/staleness.ts

Kind: added · Baseline: 8e0dc031dcfc · Target: NovaDiff · Generated: 2026-05-19T19:43:21.615Z

Overview

A new file `packages/graph-core/src/staleness.ts` adds utilities for detecting stale analysis results and merging incremental updates into a knowledge graph.

Key changes

- **Imports** – `execFileSync` from `child_process` (R1) and type imports for `KnowledgeGraph`, `GraphNode`, `GraphEdge` (R2).
- **Interface** – `StalenessResult` (R4-R7) exposes `stale: boolean` and `changedFiles: string[]`.
- **getChangedFiles** (R13-R28) runs `git diff <lastCommitHash>..HEAD --name-only` in `projectDir`, returning a trimmed list of changed file paths or an empty array if the command fails.
- **isStale** (R34-R42) calls `getChangedFiles` and flags the graph as stale when any files changed.
- **mergeGraphUpdate** (R54-R90)
 1. Builds a `Set` of changed file paths.
 2. Removes nodes whose `filePath` matches a changed file (R63-R68).
 3. Filters edges whose source or target node was removed (R75-R78).
 4. Adds new nodes/edges and updates `project.gitCommitHash` and `project.analyzedAt` (R80-R89).

Impact

- **Correctness** – Provides a deterministic way to check staleness; however, `getChangedFiles` silently returns an empty array on `git` errors, which may mask stale states.
- **Maintainability** – Centralizes graph-update logic; future changes to node/edge removal rules can be made in one place.
- **Performance** – `execFileSync` blocks the event loop; for large repositories the diff operation may become a bottleneck.
- **Compatibility** – No existing files are modified; the module is additive and does not break current APIs.

Risks & follow-ups

1. **Git failure handling** – Verify that `getChangedFiles` correctly reports errors and that `isStale` behaves as intended when `git` is unavailable.
2. **Node `filePath` undefined** – Ensure all nodes in the graph have a `filePath`; otherwise, `mergeGraphUpdate` may leave stale nodes.

3. **Large graph memory usage** – Benchmark `mergeGraphUpdate` on the largest expected graphs to confirm acceptable memory and CPU consumption.
4. **Integration** – Confirm that callers now use `isStale / mergeGraphUpdate` instead of older manual merge logic, avoiding duplicate work.